

实验 7 请求调页存储管理

7.1 实验目的

- (1) 理解页面、页表、地址转换和页面置换的概念；
- (2) 理解请求调页系统的原理；
- (3) 掌握页面置换算法的实现方法。

7.2 预备知识

常用的页面置换算法有以下几种：

1. 先进先出法 (FIFO)

这是最早出现的置换算法。该算法总是淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面予以淘汰。该算法实现简单只需把一个进程已调入内存的页面，按先后次序链接成一个队列，并设置一个指针，称为替换指针，使它总是指向最老的页面。

2 最佳置换法 (OPT)

它是由 Belady 于 1966 年提出的一种理论上的算法。其所选择的被淘汰页面，将是以后永不使用的或许是在最长(未来)时间内不再被访问的页面。采用最佳置换算法，通常可保证获得最低的缺页率。但由于人目前还无法预知一个进程在内存的若干个页面中，哪一个页面是未来最长时间内不再被访问的，因而该算法是无法实现的，但可以利用此算法来评价其它算法。

3 最近最久未使用置换法 (LRU)

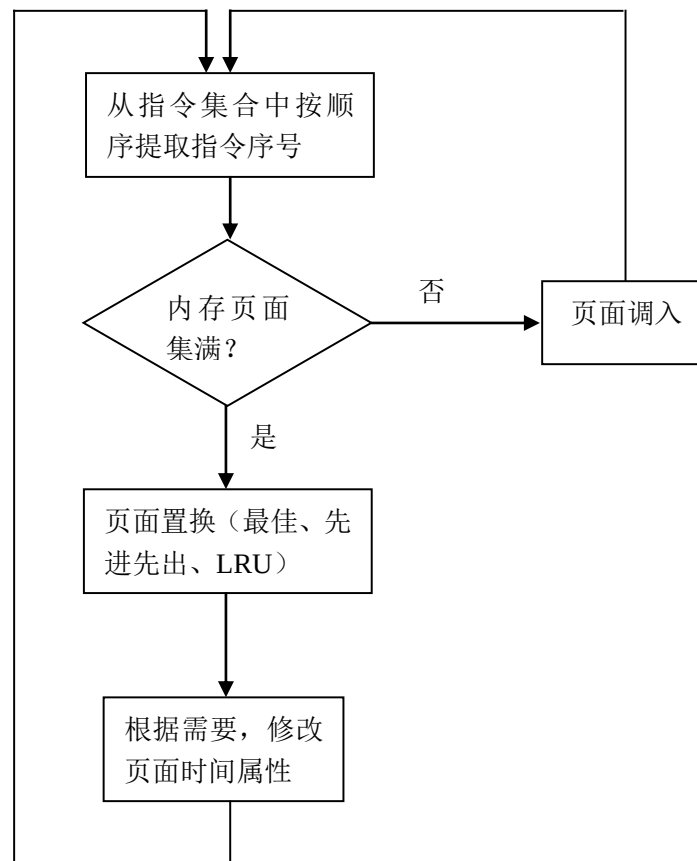
最近最久未使用 (LRU) 置换算法，是根据页面调入内存后的使用情况进行决策的。由于无法预测各页面将来的使用情况，只能利用“最近的过去”作为“最近的将来”的近似，因此，LRU 置换算法是选择最近最久未使用的页面予以淘汰。该算法赋予每个页面一个访问字段，用来记录一个页面自上次被访问以来所经历的时间 t_i ，当须淘汰一个页面时，选择现有页面中其 t 值最大的，即最近最久未使用的页面予以淘汰。

7.3 实验内容

模拟实现先进先出页面置换、最佳页面置换、LRU 页面置换算法中的一种或多种。

7.4 实验指导

1. 页面调入和置换流程



2. 结构说明

```
#define MemPageCount 3           //内存物理块数目
#define InstructionCount 20      //指令页面数目
```

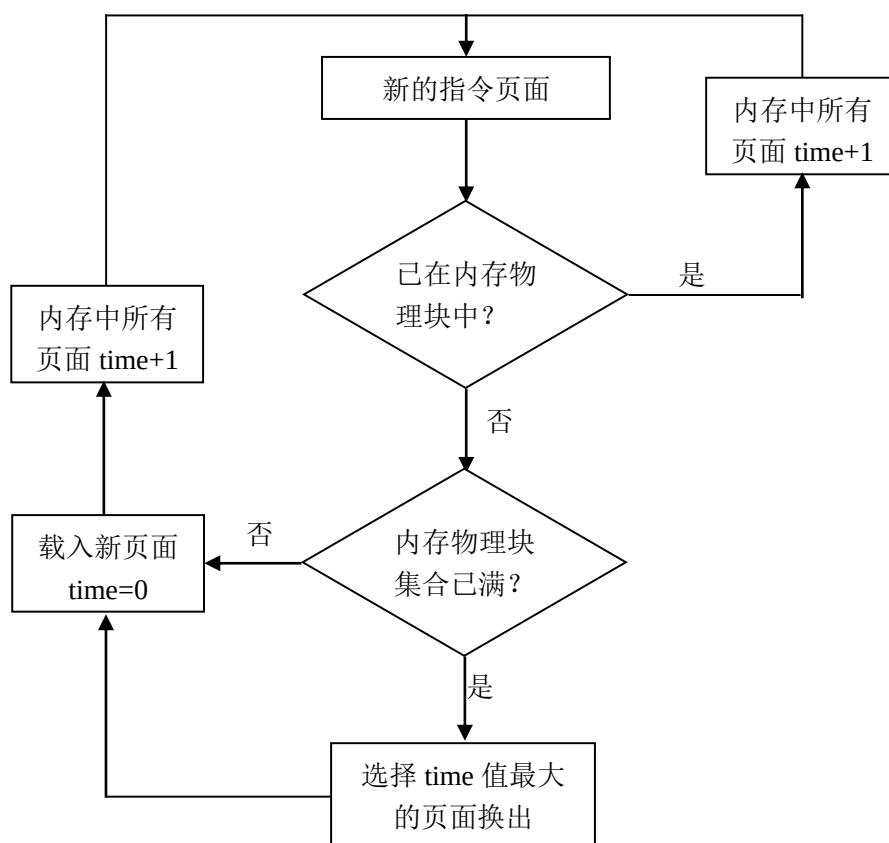
```
struct page{
    int serial;                  //页面号
    int time;                    //时间计数
}mempage[MemPageCount];
```

- MemPageCount 定义为内存中物理块数目，参考教材 5.3 的内容
- InstructionCount 定义为指令页面数目，即模拟算法中页面号引用串的数目
- mempage[MemPageCount]表示内存物理块集合，即存放指令页面的物理块
- 结构体 page 中，serial 表示页面号，time 表示时间计数

关于 time 值的说明：

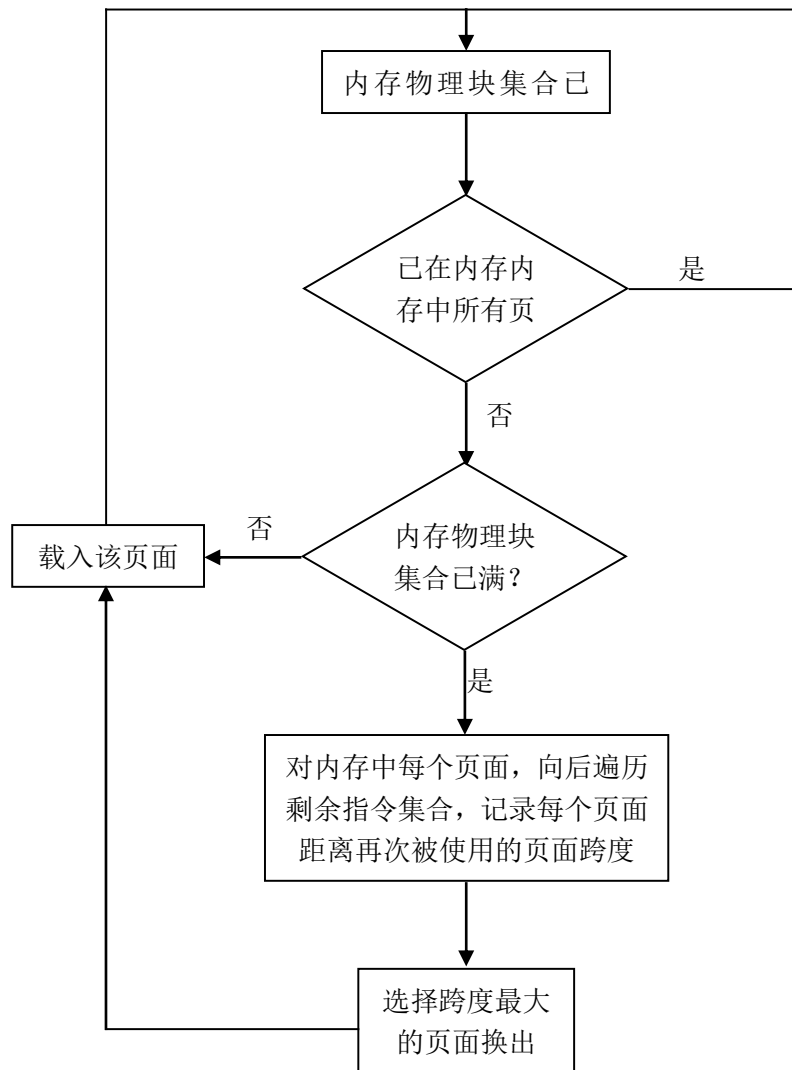
- time 字段在 OPT 算法中没有用处，而是使用 distance 来表示将来最久不使用。
- 在 FIFO 算法中，根据 time 值选择要置换出去的页面，哪个页面的 time 值越大，表示进入内存的时间越早；新进入内存的页面 time=0，然后每次都给内存中所有页面的 time 值加一，保证最先进入的页面 time 值最大，而不管该页面有否再次被使用。
- 在 LRU 算法的置换中，同样选择 time 值大的页面换出，此时 time 值越大，表示在过去越久没有被使用过；新进入的页面 time=0，然后每次给内存中所有页面的 time 值加一；如果内存中的页面被再次使用，则先将该页面的 time 置零，表示最近刚刚被使用过，保持 time 值最小，然后给所有页面的 time 值加一。

1.先进先出（FIFO）置换算法



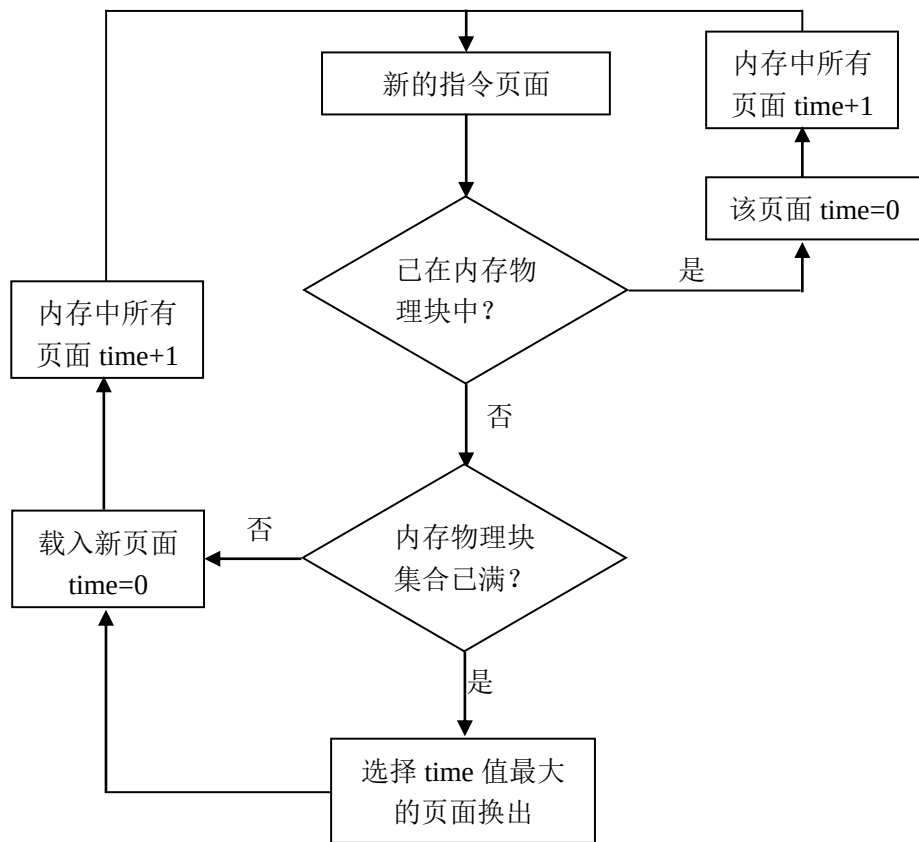
注：在 FIFO 算法中，无论有无发生缺页或者置换，都需要对每个在内存中的页面的 time 值进行增加操作，以保持最先进入的那个页面的 time 值是最大的；一个新进来的页面，其 time 值设置为 0。算法也可以通过队列结构来实现，利用队列的先进先出（FIFO）特性完成，无需设置 time 字段。

2.最佳（OPT）置换算法



注：distance 用于记录内存物理块集合中每个页面距离再次被使用的页面跨度，缺省值为 9999，如果某个页面在后续指令集合中不再出现，则用最大值 9999 缺省取代；如果页面再次被使用，则两次使用所跨的页面数，为页面跨度。用最大页面跨度表示以后永不使用或未来最长时间内不再被访问。

3.最近最久未使用（LRU）算法



注：在 LRU 算法中，无论是否发生缺页或者置换，除了命中（刚刚被访问过的页面）页面 **time** 值清零之外，其它所有内存中的页面的 **time** 值都加一，以保证最近刚刚被访问的页面的 **time** 值最小，相应 **time** 值最大的页面就是最近最久没有被访问的页面。

7.5 参考源代码

```
/******FIFO 算法*****/  
#include<stdio.h>  
  
#define MemPageCount 3          //内存物理块数目  
#define InstructionCount 21     //指令页面数目  
  
struct page{  
    int serial;                //页面号  
    int time;                  //时间计数  
}mempage[MemPageCount];  
int main(){  
    int i,j;  
    //指令页面集合,可以考虑让页面指令集合随机生成  
    int instruction[InstructionCount]={7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,1,7,0,1};  
    int mem_counter;           //内存页面集合计数器  
    int switch_counter;        //置换次数  
    int switch_flag;           //置换的位置标志  
    int lost_counter;          //缺页次数  
    int exist_flag;            //判断是否已经在内存中的标志  
    mem_counter=0;  
    switch_counter=0;  
    lost_counter=0;  
    switch_flag=0;  
    exist_flag=0;  
    for(i=0;i<MemPageCount;i++){ //内存页面集合,初始化为-1  
        mempage[i].serial=-1;  
        mempage[i].time=0;  
    }  
    //依次调入新的指令页面,如果内存页面满且新页面不存在,则置换页面  
    for(i=0;i<InstructionCount;i++){  
        for(j=0;j<MemPageCount;j++){ //判断是否已在内存中  
            if(mempage[j].serial==instruction[i]){  
                exist_flag=1;  
                break;  
            }  
        }  
    }  
}
```

```

//如果内存中没有该页面，执行缺页和置换步骤
if(exist_flag!=1){
    //内存页面集合未满,缺页不置换
    if(mem_counter<MemPageCount){
        mempage[mem_counter].serial=instruction[i];
        mempage[mem_counter].time=0;
        mem_counter=mem_counter+1;
    }
    else{          //缺页且发生置换
        switch_flag=0;
        //取得最先进入的页面在 mempage 中的位置
        for(j=1;j<mem_counter;j++){
            if(mempage[switch_flag].time<mempage[j].time)
                switch_flag=j;
        }
        mempage[switch_flag].serial=instruction[i]; //置换页面
        mempage[switch_flag].time=0;
        switch_counter=switch_counter+1;//置换次数
    }
    lost_counter=lost_counter+1;//缺页次数
    for(j=0;j<mem_counter;j++){ //打印 mempage 的情况
        printf("%d\t",mempage[j].serial);
    }
    printf("\n");
} //end if(exist_flag!=1)
for(j=0;j<mem_counter;j++){ //所有页面时间加一
    mempage[j].time=mempage[j].time+1;
}
exist_flag=0;
}
printf("缺页次数: %d 次\n",lost_counter);
printf("置换次数: %d 次\n",switch_counter);

return 0;
}

```

```

/*****OPT 算法*****/
#include<stdio.h>

#define MemPageCount 3    //内存物理块数目
#define InstructionCount 20    //指令页面数目

struct page{
    int serial;            //页面号
    int time;              //时间计数
}mempage[MemPageCount];

int main(){
    int i,j,k;
    //指令页面集合,可以考虑让页面指令集合随机生成
    int instruction[InstructionCount]={7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
    int mem_counter;        //内存页面集合计数器
    int switch_counter;     //置换次数
    int switch_flag;        //置换的位置标志
    int lost_counter;       //缺页次数
    int exist_flag;         //判断是否已经在内存中的标志
    int distance[MemPageCount]; //用于标记内存中已有页面哪个是最久的将来不使用的
    mem_counter=0;
    switch_counter=0;
    lost_counter=0;
    switch_flag=0;
    exist_flag=0;
    for(i=0;i<MemPageCount;i++){//内存页面集合,初始化为-1
        mempage[i].serial=-1;
        mempage[i].time=0;
    }
    //依次调入新的指令页面, 如果内存页面满且新页面不存在, 则置换页面
    for(i=0;i<InstructionCount;i++){
        for(j=0;j<mem_counter;j++){    //判断是否已在内存中
            if(mempage[j].serial==instruction[i]){
                exist_flag=1;
                break;
            }
        }
    }
}

```



```

if(exist_flag!=1){ //如果内存中没有该页面，执行缺页和置换步骤
    if(mem_counter<MemPageCount){ //内存页面集合未满,缺页不置换
        mepage[mem_counter].serial=instruction[i];
        mepage[mem_counter].time=0;
        mem_counter=mem_counter+1;
    }
    else{ //缺页且发生置换
        for(j=0;j<MemPageCount;j++){ //初始化距离为最大值
            distance[j]=9999;
        }
        for(j=0;j<MemPageCount;j++){
            //记录下内存页面中，再次被使用的距离
            for(k=i+1;k<InstructionCount;k++){
                if(instruction[k]==mepage[j].serial){
                    distance[j]=k;
                    break;
                }
            }
        }
        switch_flag=0;
        //寻找未来最长时间不被使用的页面在 mepage 中的位置
        //即寻找 distance 最大值
        for(k=1;k<MemPageCount;k++){
            if(distance[switch_flag]<distance[k])
                switch_flag=k;
        }
        mepage[switch_flag].serial=instruction[i];
        switch_counter=switch_counter+1;
    }
    lost_counter=lost_counter+1;
    for(j=0;j<mem_counter;j++){ //打印 mepage 的情况
        printf("%d\t",mepage[j].serial);
    }
    printf("\n");
} //end if(exist_flag!=1)
exist_flag=0;
}
printf("缺页次数: %d 次\n",lost_counter);
printf("置换次数: %d 次\n",switch_counter);

return 0;
}

```

```

/*****LRU 算法*****/
#include<stdio.h>

#define MemPageCount 3          //内存物理块数目
#define InstructionCount 20     //指令页面数目

struct page{
    int serial;                //页面号
    int time;                  //时间计数
}mempage[MemPageCount];

int main(){
    int i,j;
    //指令页面集合,可以考虑让页面指令集合随机生成
    int instruction[InstructionCount]={7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
    int mem_counter;          //内存页面集合计数器
    int switch_counter;       //置换次数
    int switch_flag;          //置换的位置标志
    int lost_counter;         //缺页次数
    int exist_flag;           //判断是否已经在内存中的标志
    mem_counter=0;
    switch_counter=0;
    lost_counter=0;
    switch_flag=0;
    exist_flag=0;
    for(i=0;i<MemPageCount;i++){//内存页面集合,初始化为-1
        mempage[i].serial=-1;
        mempage[i].time=0;
    }
    //依次调入新的指令页面, 如果内存页面满且新页面不存在, 则置换页面
    for(i=0;i<InstructionCount;i++){
        for(j=0;j<mem_counter;j++){    //判断是否已在内存中
            if(mempage[j].serial==instruction[i]){
                exist_flag=1;
                mempage[j].time=0; //如果在内存中, time 值置为零, 表示刚刚被使用过
                break;
            }
        }
    }
}

```

```

if(exist_flag!=1){ //如果内存中没有该页面，执行缺页和置换步骤
    if(mem_counter<MemPageCount){ //内存页面集合未满,缺页不置换
        mepage[mem_counter].serial=instruction[i];
        mepage[mem_counter].time=0;
        mem_counter=mem_counter+1;
    }
    else{ //缺页且发生置换
        switch_flag=0;
        for(j=1;j<mem_counter;j++){ //取得最久未被使用的页面在 mepage 中的位置
            if(mepage[switch_flag].time<mepage[j].time)
                switch_flag=j;
        }
        mepage[switch_flag].serial=instruction[i];
        mepage[switch_flag].time=0;
        switch_counter=switch_counter+1;
    }
    lost_counter=lost_counter+1;
    for(j=0;j<mem_counter;j++){ //打印 mepage 的情况
        printf("%d\t",mepage[j].serial);
    }
    printf("\n");
} //end if(exist_flag!=1)
for(j=0;j<mem_counter;j++){ //页面时间加一
    mepage[j].time=mepage[j].time+1;
}
exist_flag=0;
}
printf("缺页次数: %d 次\n",lost_counter);
printf("置换次数: %d 次\n",switch_counter);

return 0;
}

```